Module 06

Dockerfile Advance

DevOps end to end services & solutions

Agenda

- Dockerfile Advance
- Docker Advance commands
- Hands-on

Dockerfile – Port set from environment

```
FROM alpine:3.19
RUN apk add --no-cache
python3
WORKDIR /app
COPY index.html .
ENV PORT=5022
EXPOSE 5022

CMD ["sh", "-c", "python3 -m

http.server $PORT"]
```

FROM alpine:3.19

- Uses **Alpine Linux** as the base image
- Very small (~7 MB)
- Good for simple services

RUN apk add --no-cache python3

- Installs **Python 3** inside the image
- `--no-cache` keeps the image small
- Required to run the HTTP server

Dockerfile – Port set from environment

```
FROM alpine:3.19
RUN apk add --no-cache
python3
WORKDIR /app
COPY index.html .
ENV PORT=5022
EXPOSE 5022

CMD ["sh", "-c", "python3 -m

http.server $PORT"]
```

WORKDIR /app

- Sets /app as the **working directory**
- All next commands run from /app
- Creates the directory if it doesn't exist

COPY index.html .

- Copies `index.html` from your host
- Places it inside /app in the image
- This is what the HTTP server will serve

Dockerfile – Port set from environment

```
FROM alpine:3.19
RUN apk add --no-cache python3
WORKDIR /app
COPY index.html .
ENV PORT=5022
EXPOSE 5022

CMD ["sh", "-c", "python3 -m

http.server $PORT"]
```

ENV PORT=5022

- Sets an **environment variable**
- Default port the app will listen on
- Can be overridden at runtime:

EXPOSE 5022

- container listens on port **5022**

Dockerfile – Port set from environment

```
FROM alpine:3.19
RUN apk add --no-cache
python3
WORKDIR /app
COPY index.html .
ENV PORT=5022
EXPOSE 5022

CMD ["sh", "-c", "python3 -m

http.server $PORT"]
```

CMD ["sh", "-c", "python3 -m http.server

$PORT"]

- Command that runs when the container starts
- Starts Python's built-in HTTP server
- Uses `$PORT` from the environment

Docker Build and Run

Docker top ‹ C ID ›

It shows you **what processes are actually running inside a container**

Docker stats ‹ C ID ›

live performance dashboard for containers

It shows **real-time resource usage** — CPU, memory, network, and disk I/O

Docker Build and Run

Docker inspect ‹ C ID ›

deep-dive X-ray of Docker

It shows **everything Docker knows** about an image or container — config, networking, mounts, env vars, ports, PID, etc

Docker logs –follow ‹ C ID ›

watch a container's output live

Like less

Docker logs < C ID> | less

Hands-On – Docker Argument port and Jenkins

- Go to Repo – Docker Advance commands docker top, stats, inspect, etc

Questions?

Thank you for participating.

Feel free to reach out:
https://dinghy-e2e.com